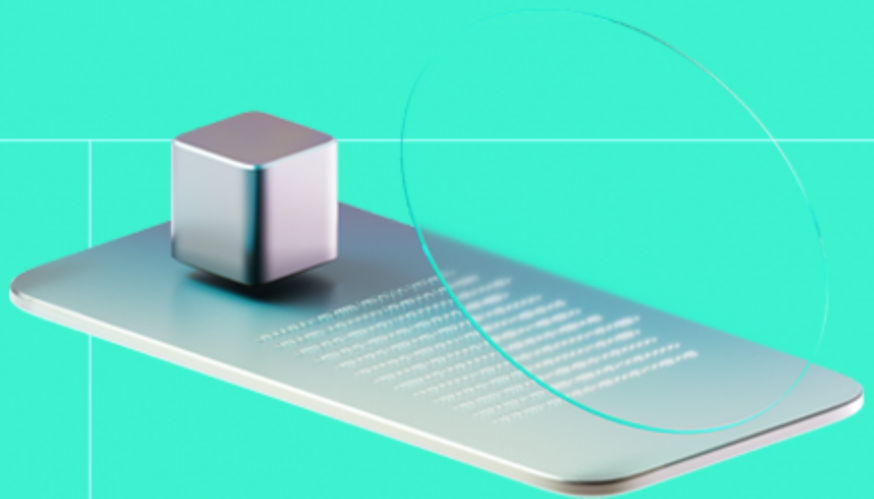




Smart Contract Code Review And Security Analysis Report

Customer: LTO Network

Date: 1/10/2025



We express our gratitude to the LTO Network team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

EQTY is a capped ERC20 token with a maximum supply of 500 million, featuring burnability and controlled minting restricted to a designated bridge wallet within a limited timeframe.

Document

Name	Smart Contract Code Review and Security Analysis Report for LTO Network
Audited By	Panagiotis Konstantinidis, Ataberk Yavuzer
Approved By	Ivan Bondar
Website	N/A
Changelog	1/10/2025 - Preliminary Report
Platform	Base
Language	Solidity
Tags	Fungible Token, ERC-20
Methodology	https://hackenio.cc/sc_methodology

Review Scope

Contract	https://basescan.org/token/0xc71f37d9bf4c5d1e7fe4bccb97e6f30
Address	b11b37d29

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

2	0	0	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	0
Low	0

Vulnerability	Severity Status	
F-2025-13212 - Identical Token Name and Symbol May Lead to Confusion	Info	Pending Fix
F-2025-13213 - Misleading Constant Name For Supply Cap	Info	Pending Fix

Documentation quality

- Functional and technical requirements are specified in the NatSpec comments.
- Deployment and testing procedures are not included.

Code quality

- The code leverages OpenZeppelin contracts for ERC-20, burnable, and capped supply functionality.
- Implementation is minimal and well-structured, without unnecessary complexity.
- No redundant logic or template leftovers were found.

Test coverage

Code coverage of the project is **0%**.

- No test files are implemented.
- The test environment is not configured.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	7
Findings	8
Vulnerability Details	8
Disclaimers	12
Appendix 1. Definitions	13
Severities	13
Potential Risks	13
Appendix 2. Scope	14
Appendix 3. Additional Valuables	15

System Overview

EQTY is a simple ERC-20 token with a maximum supply of 500 million tokens. At deployment, a bridge wallet and a minting deadline are set. Only that wallet can mint, and only until the specified deadline is reached, after which minting is permanently disabled and the cap is fixed. Holders can also burn their tokens at any time to permanently reduce supply. Beyond these rules, EQTY behaves as a standard ERC-20 token, which is using 18 decimals, and is compatible with common wallets and DEXs, with no other ongoing admin role or upgrade controls defined in the current contract.

It has the following attributes:

- **Name:** EQTY
- **Symbol:** EQTY
- **Decimals:** 18
- **Maximum supply cap:** 500,000,000 tokens

Privileged roles

- The `bridgeWallet`, defined at deployment, holds the authority to mint EQTY tokens up to the fixed maximum cap of 500 million. This privilege is strictly time-bound and expires once the configured `mintDeadline` is reached, after which no additional minting is permitted.

Potential Risks

- **Centralized Control of Minting Process:** Until the configured `mintDeadline`, all token issuance is solely controlled by the `bridgeWallet`. Compromise or misuse of this single key could trigger unintended mints (until the 500 Million cap is reached), and the chosen deadline directly determines how long this centralization persists.
- **Insufficient Multi-Signature Controls for Critical Functions:** The mint function is gated by a single wallet address. Lack of multisig increases the risk of erroneous or malicious mint operations prior to the deadline.
- **Absence of Time-lock Mechanisms for Mint Actions:** There is no enforced delay or review window for mint transactions. If the `bridgeWallet` key is compromised, large mints can occur immediately, until the maximum cap is reached.
- **Single Point of Failure and Control:** Until the `mintDeadline`, supply expansion depends solely on the `bridgeWallet`. Key loss, compromise, or operator error could adversely impact token distribution and market trust.

Findings

Vulnerability Details

F-2025-13212 - Identical Token Name and Symbol May Lead to Confusion - Info

Description:

The `EQTY` contract defines identical token name and symbol as `EQTY` in the `constructor`. While this may be intentional, it can lead to confusion during deployment and interaction, as multiple tokens with identical identifiers complicate differentiation across tools, wallets, or explorers.

```
constructor(  
    address _bridgeWallet,  
    uint256 _mintDeadline  
)  
  
    ERC20("EQTY", "EQTY")  
    //other code  
}
```

Users often rely on the symbol as a concise identifier and the name for a descriptive label. Having both identical might obscure the token's purpose or identity.

Assets:

- `EQTY.sol`
[<https://basescan.org/token/0xc71f37d9bf4c5d1e7fe4bccb97e6f30b11b37d29#code>]

Status:

Pending Fix

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: To improve clarity and usability, consider using a descriptive `Name` and a shorter, unique `Symbol` in the `EQTY` contract.

F-2025-13213 - Misleading Constant Name For Supply Cap - Info

Description:

The EQTY contract leverages OpenZeppelin's `ERC20Capped`, which sets a maximum mintable supply. In the code, this value is stored in a constant named `TOTAL_SUPPLY`:

```
uint256 private constant TOTAL_SUPPLY = 500_000_000 ether; // 500 million tokens
```

In general, the total supply naming in ERC-20 contracts refers to the amount already minted and in existence. Naming the cap as `TOTAL_SUPPLY` blurs that distinction and implies all 500M tokens already exist. In reality, this constant represents the upper limit that will be minted, not the current circulating amount.

As a result, this mismatch can mislead users or analytics (e.g., dashboards, tools) that might read `TOTAL_SUPPLY` value from the code, as the circulating supply instead of a cap.

Assets:

- EQTY.sol
[<https://basescan.org/token/0xc71f37d9bf4c5d1e7fe4bccb97e6f30b11b37d29#code>]

Status:

Pending Fix

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Rename the constant value to reflect its true purpose as a maximum supply cap, rather than implying a pre-minted supply. A clearer naming convention improves readability and reduces misinterpretation risk.

Example Fix:

```
// Max supply cap enforced by ERC20Capped (not pre-minted)
uint256 private constant MAX_SUPPLY_CAP = 500_000_000 ether;

constructor(address _bridgeWallet, uint256 _mintDeadline)
    ERC20("EQTY", "EQTY")
    ERC20Capped(MAX_SUPPLY_CAP)
{ /* ... */ }
```

Additionally, consider adding a NatSpec comment explicitly clarifying that this value is the maximum supply enforced by `ERC20Capped`, not the circulating `totalSupply()`.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Contract Address	https://basescan.org/token/0xc71f37d9bf4c5d1e7fe4bccb97e6f30b11b37d29
Whitepaper	N/A
Requirements	N/A
Technical Requirements	N/A

Asset	Type
EQTY.sol	Smart
[https://basescan.org/token/0xc71f37d9bf4c5d1e7fe4bccb97e6f30b11b37d29#code] Contract	

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.